

Comparison of speed and performance between Python and R implementations of the AddiVortes algorithm

Lewis Maltby

July 12, 2026

1 Introduction

The purpose of this article is to compare the speed and performance of the AddiVortes algorithm between the current up-to-date Python and R implementations. We will benchmark the fitting time of a model, as well as the amount of time required to make simple predictions on a test set, to try to determine which of the two implementations is faster to use. We will also benchmark the in-sample and out-of-sample RMSE on that data to determine which of the two implementations gives predictions with lower errors.

The code used to generate the benchmarking data, as well as the generated data itself and all referenced files, are available to view at <https://github.com/snowy-the-leopard/AddiVortes>. This data was generated using the ADDIVORTES R package, version 0.6.6, available to view here <https://github.com/johnpaulgosling/AddiVortes>, and the ADDIVORTES Python package, version 0.6.6, available to view here <https://github.com/johnpaulgosling/py-addivortes>.

2 Benchmarking methodology

To ensure fairness, consistency, and accuracy with these tests, the following controls were implemented on these benchmarks:

1. The benchmarks were run on a new GitHub Codespace, a clean installation of Python and R were used, with only the necessary packages downloaded to minimise background resource usage. The Codespace used is a 2-core 2.8GHz system with 8GB of memory. Exact details of hardware used can be found at `AddiVortes\mySpecs.html`.
2. The code was set up such that each implementation used only a single thread, to counteract the potentially different methods of multi threading between the two languages. This choice was made as we are working under the assumption that when we are working with HPC resources, the code will be optimised to allow multiple threads to be used at once. This choice allows us to compare algorithmic efficiency in each implementation more directly.
3. The same models and predictions were fitted repeatedly in order to capture the small variations in fit and prediction times between runs. This will also produce a set of slightly varying prediction results due to the random nature of the MCMC fitting algorithm, and allow us to better analyse how consistent our results are.
4. The benchmarks were run sequentially rather than simultaneously to minimise the risk of shared resources skewing the results, where one script could take computational priority over another, leading to a larger discrepancy in results.
5. The timings were taken only between the computation required for fitting and generating predictions, with the pre-generated datasets already loaded in prior to this.

3 Datasets used in benchmarking

To allow for a wide scope of comparison between the two implementations, the following 4 different datasets were used in benchmarking:

3.1 Boston dataset

The Boston Housing Dataset, derived from information from the U.S. Census Service. This consists of 13 numerical covariates, and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`

3.2 Synthetic dataset

We have constructed a synthetic dataset using noisy functions of independently and identically distributed $U[0, 1]$ observations. This consists of 5 numerical covariates, and we model it by taking all the parameters as their default values.

3.3 Spherical dataset

We have constructed random longitudes and latitudes to represent positions on a sphere, and a noisy response function. This consists of 2 numerical covariates (the coordinates), and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`, `totalMCMCiter = 500`, `mcmcBurnIn = 100`, `metric = "S"`. Note that this uses great circle distance, the shortest distance between two points over the surface of the sphere.

3.4 Categorical dataset

We have constructed this dataset using random numerical and categorical covariates, as well as a noisy response function. This consists of 2 numerical and 2 categorical covariates, and we model it using the following fixed parameters, with the rest of the parameters taking their default values: `m = 50`, `totalMCMCiter = 500`, `mcmcBurnIn = 100`, `catScaling = 1`

4 Benchmark results

The full benchmarks, which carry out model fitting and prediction on the above 4 datasets, were run for 106 iterations each. The code can be found at `AddiVortes\tests.R` and `AddiVortes\tests.py`

Table 1: Mean Fit and Prediction Times (R vs Python)

Test	Fit Mean (R)	Fit Mean (Python)	Fit % Diff	Pred Mean (R)	Pred Mean (Python)	Pred % Diff
Test 1	13.876	11.177	-19.452	8.091	3.844	-52.490
Test 2	1.745	1.253	-28.191	1.371	0.656	-52.126
Test 3	3.312	3.045	-8.059	2.365	1.973	-16.558
Test 4	0.616	0.402	-34.768	0.497	0.208	-58.028

Table 2: Standard Deviation of Fit and Prediction Times (R vs Python)

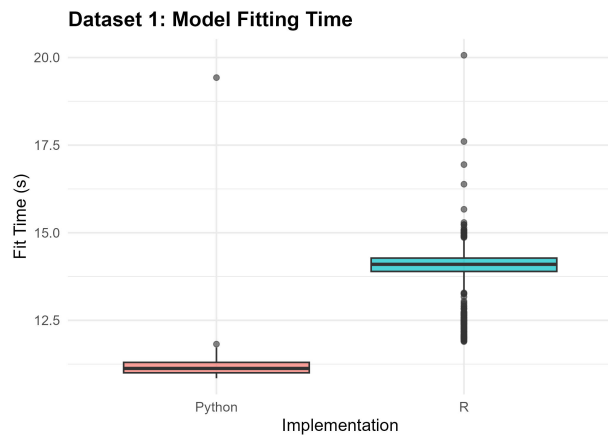
Test	Fit SD (R)	Fit SD (Python)	Fit % Diff	Pred SD (R)	Pred SD (Python)	Pred % Diff
Test 1	0.879	0.410	-53.396	0.798	0.090	-88.720
Test 2	0.243	0.129	-46.780	0.147	0.037	-74.933
Test 3	0.262	0.270	3.057	0.233	0.182	-21.905
Test 4	0.053	0.075	39.904	0.044	0.030	-32.078

Table 3: Mean In-sample and Out-of-sample RMSE (R vs Python)

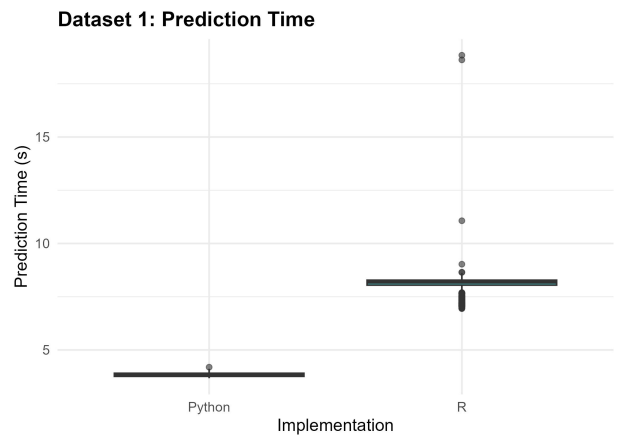
Test	iRMSE Mean (R)	iRMSE Mean (Python)	iRMSE % Diff	oRMSE Mean (R)	oRMSE Mean (Python)	oRMSE % Diff
Test 1	0.784	0.794	1.245	3.667	3.677	0.289
Test 2	1.021	1.081	5.833	1.353	1.420	4.909
Test 3	1.219	1.240	1.769	2.202	2.217	0.656
Test 4	2.237	2.244	0.310	3.030	3.037	0.221

Table 4: Standard Deviation of RMSE (R vs Python)

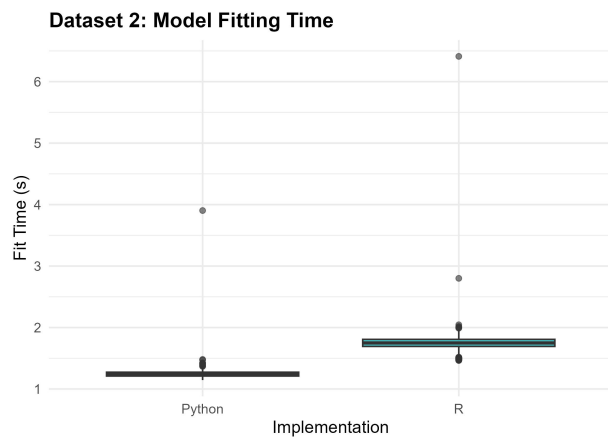
Test	iRMSE SD (R)	iRMSE SD (Python)	iRMSE % Diff	oRMSE SD (R)	oRMSE SD (Python)	oRMSE % Diff
Test 1	0.032	0.036	10.720	0.103	0.115	11.999
Test 2	0.084	0.082	-3.255	0.114	0.107	-5.881
Test 3	0.060	0.059	-0.096	0.045	0.049	8.684
Test 4	0.038	0.037	-3.008	0.062	0.071	13.504



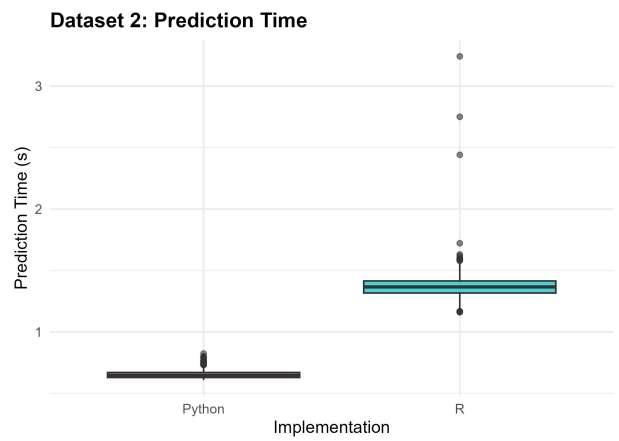
(a) Fit 1



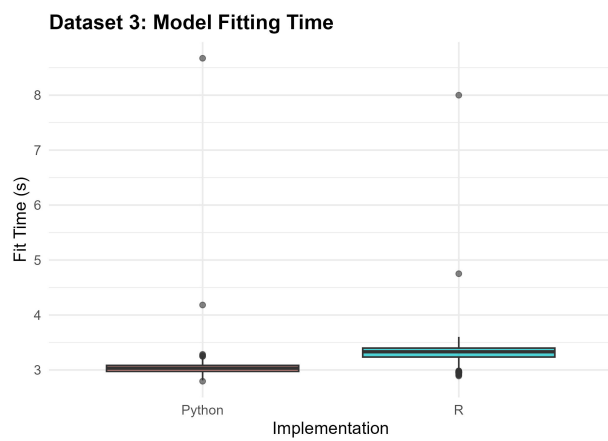
(b) Pred 1



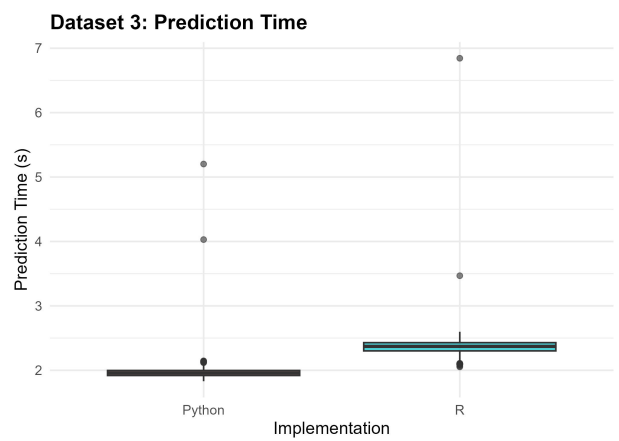
(c) Fit 2



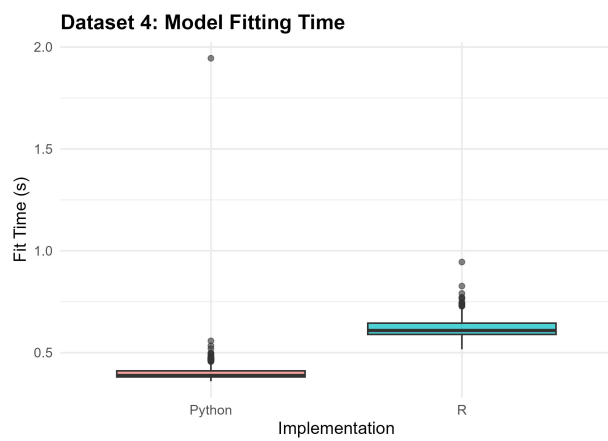
(d) Pred 2



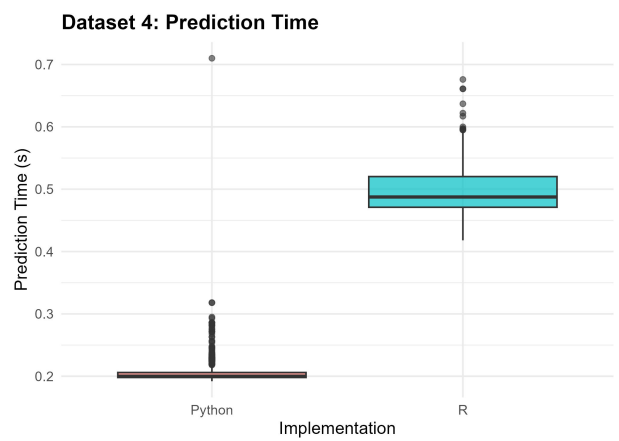
(e) Fit 3



(f) Pred 3

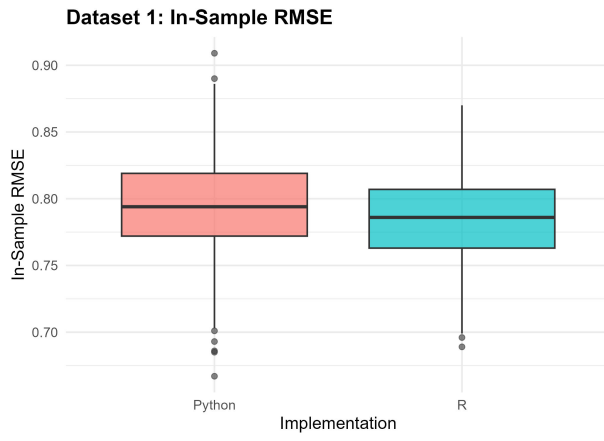


(g) Fit 4

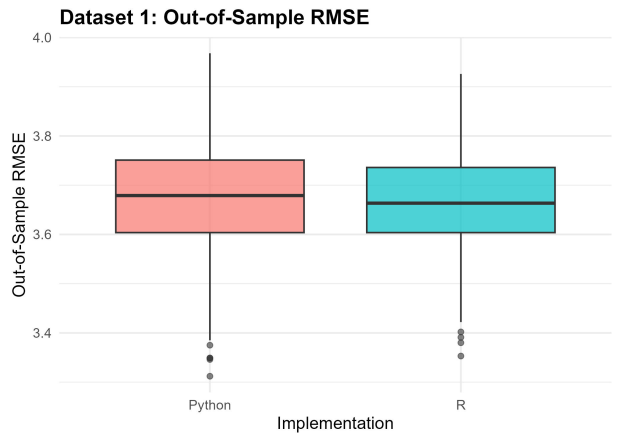


(h) Pred 4

Figure 1: Model fitting and prediction times across datasets.



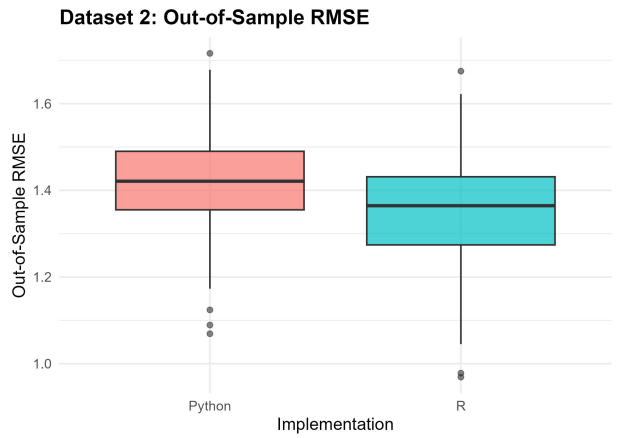
(a) iRMSE 1



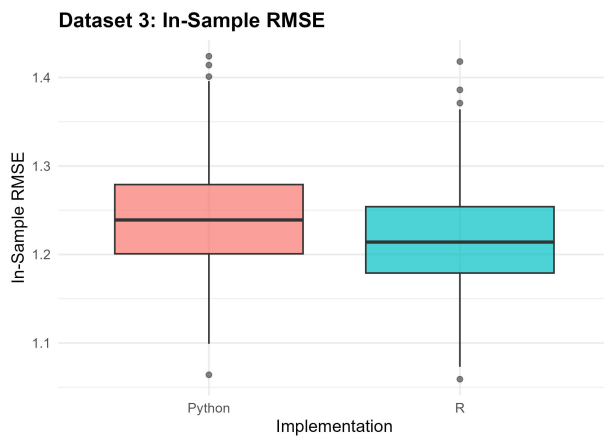
(b) oRMSE 1



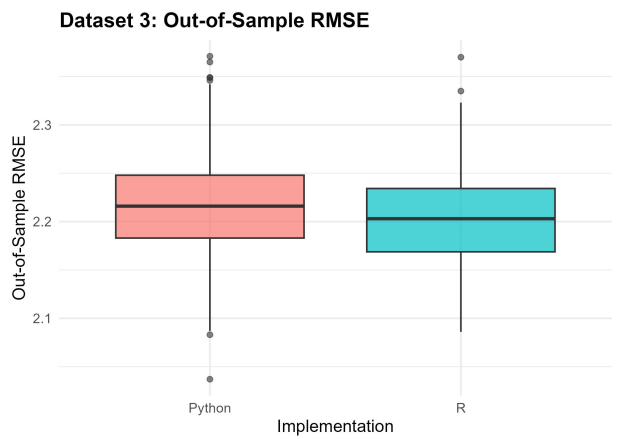
(c) iRMSE 2



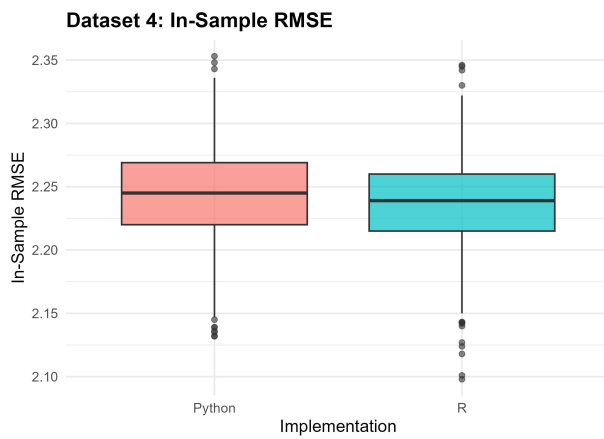
(d) oRMSE 2



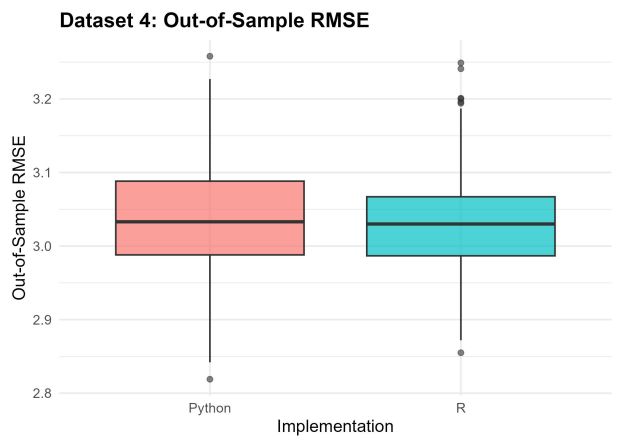
(e) iRMSE 3



(f) oRMSE 3



(g) iRMSE 4



(h) oRMSE 4

Figure 2: In-sample and out-of-sample RMSE across datasets.

5 Preliminary analysis of results

5.1 Fit times

The mean fit time using Python was substantially lower for all models. With an average decrease of 23% in mean fit time using Python, we can certainly conclude that the Python implementation is significantly faster than R at fitting the AddiVortes model using a Euclidean metric. The standard deviation of fit times using Python gave mixed results when compared to using R, but for the first 3 models Python has either roughly the same, or a much lower standard deviation for fit times.

Therefore in terms of both lower and more consistent fit times, Python is superior.

5.2 Prediction times

The mean prediction time using Python was significantly lower for all models, with an average decrease of 45% when using Python instead of R. The standard deviation of prediction times using Python was significantly lower for all models, with an average decrease of 54% when using Python instead of R.

Therefore in terms of both lower and more consistent prediction times, Python is superior.

5.3 In-sample RMSEs

The mean in-sample RMSE using Python was slightly higher for all models, with an average increase of 2% when using Python instead of R. The standard deviation of in-sample RMSEs using Python gave mixed results in comparison to R. The standard deviation of in-sample RMSEs on model 1 was significantly higher. This model, representing real world data that could be more unstructured than the synthetic datasets we generated, should be the most significant for us here, where the other differences vary in sign.

Therefore in terms of lower and more consistent in-sample RMSEs, we should take a slight preference for R, with its slightly, but consistently, better mean in-sample RMSEs, and its proficiency for more consistent in-sample RMSEs with the real-world dataset used.

5.4 Out-of-sample RMSEs

The mean out-of-sample RMSE using Python consistently gave slightly higher values, with an average increase of 2% across the 4 models. The standard deviation of out-of-sample RMSEs using Python gave mixed results, but overall seemed to be higher, with an average increase of 7% across the 4 models. This demonstrates that R generates more consistent out-of-sample RMSEs

Therefore in terms of lower and more consistent out-of-sample RMSEs, we should take a slight preference for R

6 Significance of results using standard errors

Let $\bar{X}_{f_1}$ be our sample mean fit time for test 1 using R and $\bar{Y}_{f_1}$ be our sample mean fit time for test 1 using Python.

$$Var(\bar{X}_{f_1} - \bar{Y}_{f_1}) = Var(\bar{X}_{f_1}) + Var(\bar{Y}_{f_1})$$

Note that we are using samples with 106 observations and hence need to use our standard errors, by scaling our standard deviations from tables 2 and 4 by a factor of $\frac{1}{\sqrt{n}}$. We can then calculate the standard error of the difference of the means:

$$SE(\bar{X}_{f_1} - \bar{Y}_{f_1}) = \sqrt{SE(\bar{X}_{f_1})^2 + SE(\bar{Y}_{f_1})^2}$$

We divide the difference in the mean values $\bar{X}_{f_1} - \bar{Y}_{f_1}$ by the combined standard error to give z-values for these differences. We will consider z-values over 2 as significant (roughly 95% significance level). The results of these analyses are below:

Table 5: Fit and prediction standard errors

Test	Fit SE (R)	Fit SE (Python)	Pred SE (R)	Pred SE (Python)	Fit SE (Comb)	Pred SE (Comb)
Test 1	0.085	0.040	0.078	0.009	0.094	0.078
Test 2	0.024	0.013	0.014	0.004	0.027	0.015
Test 3	0.025	0.026	0.023	0.018	0.037	0.029
Test 4	0.005	0.007	0.004	0.003	0.009	0.005

Table 6: iRMSE and oRMSE standard errors

Test	iRMSE SE (R)	iRMSE SE (Python)	oRMSE SE (R)	oRMSE SE (Python)	iRMSE SE (Comb)	oRMSE SE (Comb)
Test 1	0.003	0.003	0.010	0.011	0.005	0.015
Test 2	0.008	0.008	0.011	0.010	0.011	0.015
Test 3	0.006	0.006	0.004	0.005	0.008	0.007
Test 4	0.004	0.004	0.006	0.007	0.005	0.009

Table 7: Fit and prediction z-values

Test	Fit Diff	Pred Diff	Fit SE (Comb)	Pred SE (Comb)	Fit Diff SEs	Pred Diff SEs
Test 1	2.699	4.247	0.094	0.078	28.663	54.451
Test 2	0.492	0.714	0.027	0.015	18.417	48.470
Test 3	0.267	0.392	0.037	0.029	7.308	13.636
Test 4	0.214	0.288	0.009	0.005	24.084	56.064

Table 8: iRMSE and oRMSE z-values

Test	iRMSE Diff	oRMSE Diff	iRMSE SE (Comb)	oRMSE SE (Comb)	iRMSE Diff SEs	oRMSE Diff SEs
Test 1	0.010	0.011	0.005	0.015	2.079	0.706
Test 2	0.060	0.066	0.011	0.015	5.220	4.383
Test 3	0.022	0.014	0.008	0.007	2.637	2.223
Test 4	0.007	0.007	0.005	0.009	1.339	0.731

Clearly, all of the differences in fit times and prediction times are highly significant, with Python having the significantly lower times for both. Some of the iRMSE and oRMSE differences are significant, but there are also some insignificant results, especially within oRMSE where significances were lower than in iRMSE.

Therefore, since there is limited evidence to suggest the R implementation has lower oRMSEs, and there is strong evidence to suggest the Python implementation has faster fit and prediction times, I would suggest Python to be used for further implementation and investigation